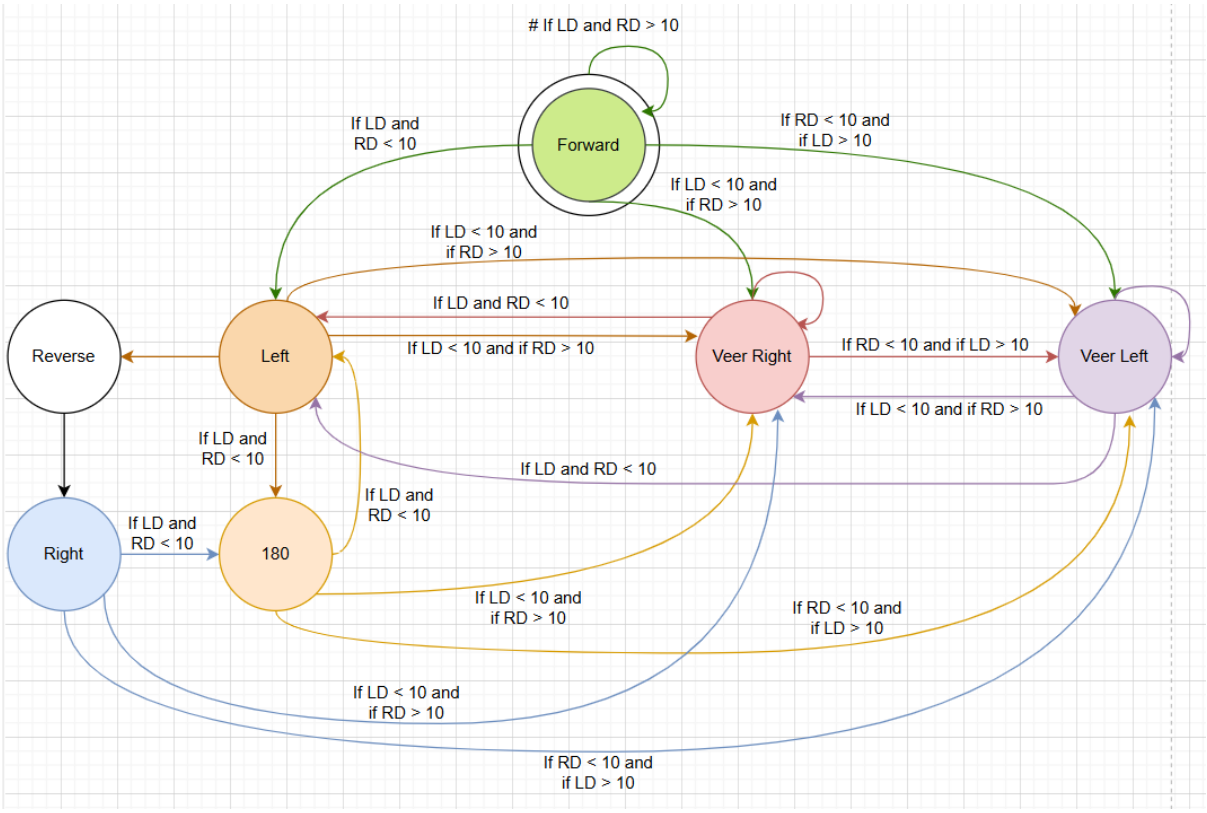
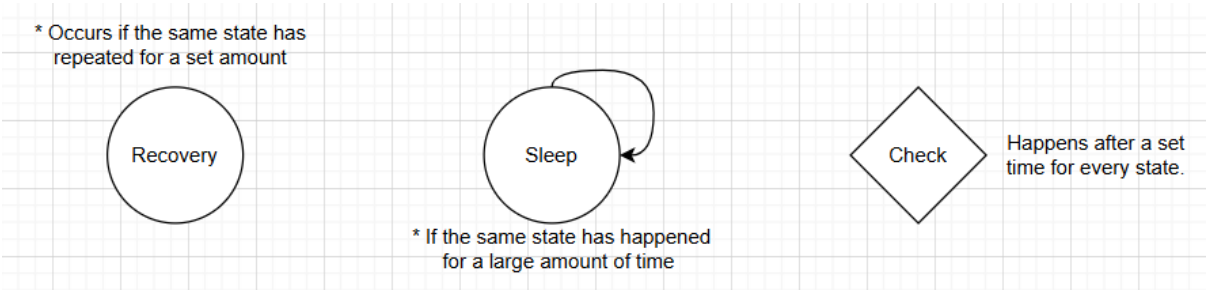
AFSM Diagram and Description





State Transition Table:

Previous State	Transition	Current State
Forward	If RD < 10 and LD > 10	Veer Left
Forward	If LD < 10 and RD > 10	Veer Right
Forward	If RD and LD < 10	Left
Forward	If currentState == prev_state and count > 3	Recovery
Forward	If LD and RD > 10	Forward
Veer Left	If LD < 10 and RD > 10	Veer Right
Veer Left	If LD and RD < 10	Left
Veer Left	If LD and RD > 10	Forward
Veer Left	If currentState == prev_state and count > 3	Recovery
Veer Left	If RD < 10 and LD > 10	Veer Left
Veer Right	If RD < 10 and LD > 10	Veer Left
Veer Right	If RD and LD < 10	Left
Veer Right	If RD and LD > 10	Forward

AFSM Diagram and Description

Veer Right	If currentState == prev_state and count > 3	Recovery
Veer Right	If LD < 10 and RD > 10	Veer Right
Left	If LD and RD < 10	180
Left	If LD and RD > 10	Forward
Left	If LD < 10 and RD > 10	Veer Right
Left	If RD < 10 and LD > 10	Veer Left
Left	If currentState == prev_state and count > 3	Recovery
180	If RD and LD < 10	Left
180	If RD and LD > 10	Forward
180	If RD < 10 and LD > 10	Veer Left
180	If LD < 10 and RD > 10	Veer Right
180	If currentState == prev_state and count > 3	Recovery
Reverse	Reverses for a set time	Right
Reverse	If currentState == prev_state and count > 3	Recovery
Right	If LD and RD < 10	180
Right	If LD and RD > 10	Forward
Right	If LD < 10 and RD > 10	Veer Right
Right	If RD < 10 and LD > 10	Veer Left
Right	If currentState == prev_state and count > 3	Recovery
Recovery	If recovery_Count > 3	Sleep
Sleep	If LD or RD < 10	Forward

State Description Table:

State	Description
Forward – Start	The forward state sets all the motors to move forward at the same speed. It is triggered if both sensors do not detect an object. This state is controlled by a timer making it move forward for a set time before checking the sensors and either continuing to move forward or changing states. If it continues to move forward the count is incremented by 1. If the state has changed, the count is set back to zero. Every state other than the reverse state can transition to the forward state. This is because it is a basic state, and the reverse state will never transition to it because that can cause a loop. For example, you move forward, both sensors detect an object, and you reverse. Now both sensors no longer detect the object, so you move forward again. The object is detected again, etc…
Veer Left	This state is the basic turn state. It is triggered if an object is detected only by the right sensor. It causes the motors on the left to slow down so the robot can do a smooth left turn, without turning on the spot.

AFSM Diagram and Description

	The timer for this state is used to set how long the motors run before checking the sensors. If there is no condition met for a state transition, the robot will continue to veer left. If a condition is met, it will move to that state. If the state does not change the count is incremented by 1. If it has changed, then the count is set back to zero.
Veer Right	This state is also a basic turn state, turning in the opposite direction as the one above. It is triggered only if the left sensor detects an object. It causes the servos to be slowed on the right side in order to create the same kind of turn described above. The timer is used for the sensor checks, where it tells the servos how long to run before completing the check. This state can loop on itself if no condition is met for a state transition, incrementing the count by 1. If a condition is met, the state is changed, and the count is set back to zero.
Left	This state is controlled by a timer that is set at a specific time to allow for a 90-degree turn. It is triggered if an object is detected by both senses. It turns 90-degrees based on a set timer. This condition does not immediately go to the reverse state on account of a possible L turn or T junction in the maze. Going directly to reverse means the robot could miss these turns entirely. This state makes the left servos go in reverse whilst keeping the right servos moving forward to do a pivot-like turn. This state should not be able to loop on itself, however, it still can transition to the recovery state as a precaution for possible errors that may occur.
180	This state allows the robot to completely flip itself around. It is its own state as it turns for a specific time that is larger than the basic 90-degree turns. This state also causes the robot to pivot, by reversing the left servos whilst keeping the right ones moving forward. It also should not loop on itself but can also transition to the recovery state as a precaution for possible errors. This state transitions back to left if both sensors, after the check, still detect an object, to straighten the robot before reversing.
Reverse	This state causes the servos to move backwards for a short time. It is immediately triggered after the second left turn, so when the robot turns 180 detects an object and turns left 90 degrees. It does not have a direct condition involving the sensors, its condition is the 180-state going back to the left turn state. This is so that it can avoid the loop of constantly reversing and moving forward, and so that it does not interfere with the left turn. In more detail, as the condition for the left turn is if both sensors detect an object, and all other conditions involving the sensors have been used, adding such a condition for this transition would interfere with another state and is not necessary for this state to function.
Right	This state is the 90-degree right turn. It operates similarly to the left turn in which it performs a pivot-like turn where it reverses the right wheels, keeping the left ones moving forward. This state performs a check of the sensors once the 90-degree turn is complete. The turn is controlled by the timer. This state is triggered immediately after the

	reverse state has ended. Once the sensors are checked, it then moves to the new state based on the sensor reading, be that looping back to the 180 state or another state. This state should also not be able to loop immediately onto itself but can still transition to the recovery state if need be.
Recovery	The recovery state is the safety state. It is essentially a prolonged check in which all the servos stop, and the sensors are read multiple times before moving to the appropriate state. As it is a safety state, all states can transition to it. The recovery state is triggered if the same state is repeated multiple times. So, if the count, mentioned before, becomes greater than 3, the recovery state is triggered. This state can transition to any state based on the sensors. It uses the timer so that it only stops for a couple of seconds before starting again.
Sleep	The sleep state is like a prolonged recovery state. It is triggered if the same state has been triggered for a long time, i.e. 20 seconds. This state assumes that the robot has either lost the maze or has solved it. Once this state is triggered, it stops the motors but keeps checking the sensors periodically. This is because this state can transition back into any state if either sensor detects an object. If this state is transitioned out of, the count is reset back to zero.

Notes:

- This AFSM diagram is a guideline for my code and may change depending on new scenarios, time constraints, and coding restrictions.
- All states use a timer to function, and they all can enter both the recovery and sleep states which is why they are starred (*) to reduce the number of arrows, so the diagram is easier to understand.
- The start state has a hashtag (#) because all states other than the reverse state can transition to it, so no arrows go into the forward state, so the diagram is neater and easier to understand.